```
#           The *S.T.A.R. Program: Predicting log_SFR_Ha Using Novel Mathematical Framework
#                       Bridging the Gap Between Astrophysics & Number Theory
#================================================================================================
================================================================================================
========
# *S.T.A.R.: Symbolically. Tuned. Astronomical. Regression
# -- S: Symbolic-Regression via gplearn
# -- T: Tuned hyperparameter optimization with Optuna
# -- A: Astronomical predictions of star formation rates in galaxies
# -- R: Regressing pipeline of HistGradientBoostingRegressor, gpLearn, LightGBM, XGBoost, & CatBoost
#================================================================================================
================================================================================================
========

import pandas as pd
import numpy as np
import shap
import matplotlib.pyplot as plt
import seaborn as sns  # Used for correlation heatmap
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.metrics import r2_score
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.ensemble import HistGradientBoostingRegressor
from gplearn.genetic import SymbolicRegressor
import optuna
import warnings
from numpy.polynomial import Polynomial  # Used for SFR trend fitting
from scipy.spatial import cKDTree  # Used for clustering
import lightgbm as lgb
import xgboost as xgb
from catboost import CatBoostRegressor
from tqdm import tqdm
from astropy.cosmology import FlatLambdaCDM
import astropy.units as u
from astropy.coordinates import SkyCoord
from astropy.constants import L_sun
from dustmaps.sfd import SFDQuery                    # Key Notes:
import multiprocessing as mp                  # Confirm all features within csv files
from functools import partial                 # confirm feature selection section with csv files
import time                                   # confirm L_cosmo(s) and Cosmic Rank features with csv files
import os                                     # complete BSD likeliness section
import pickle                                 # confirm program set for 500,000 rows (down from original 1,000,000 row
target)
                                              # Test uses cross matching csv files @ sky cone center coordinates
warnings.filterwarnings("ignore")       # (RA 200.0 + DEC 0.0 ~ with a 1 - 6 arcsec cross-match tolerance in a 15 deg.
Radius )
```

```
# List of CSV files
================================================================================================
================================================================================
csv_files = [                                   # Survey Table's
    "ALLWISE_SDSSDR16.csv",                 #1 ALLWISE + SDSS DR16
    "TwoMass_SDSSDR16.csv",                 #2 2Mass + SDSS DR16
    "STARHORSE2021_SDSSDR16.csv",     #3 STARHORSE2021 + SDSS DR16
    "GALAXGR6+7AIS_SDSSDR16.csv",     #4 GALAX GR6+7 AIS + SDSS DR16
    "UKIDSSDR9LAS_SDSSDR16.csv",       #5 UKIDDS DR9 LAS + SDSS DR16
    "GAIADR3AP_SDSSDR16.csv",             #6 GAIA DR3 Astro. Param. + SDSS DR16
    "PanST2DR1_SDSSDR16.csv",             #7 PanSTARRS-DR1 + SDSS DR16
    "PanSTDR1_SDSSDR16.csv",               #8 PanSTARRS DR1 + SDSS DR!6
    "SDSS_MangaSpec_with_fluxes.csv",     #9 SDSS DR16 + SDSS DR16 EmissionLines + MangaSpecAll
    "merged_galspec_gz2.csv"                 #10 SDSS DR16 GalSpecAll + Galaxy Zoo 2
]

# Column mappings for each file
================================================================================================
================================================================================
column_mappings = {
    "ALLWISE_SDSSDR16.csv": [
        'objID', 'RA_ICRS', 'DE_ICRS', 'umag', 'gmag', 'rmag', 'imag', 'zmag', 'e_umag', 'e_gmag',
        'e_rmag', 'e_imag', 'e_zmag', 'Jmag', 'Hmag', 'Kmag', 'e_Jmag', 'e_Hmag', 'e_Kmag', 'zsp'
  ],# ------------------------------------------------------------------------------------------
    "TwoMass_SDSSDR16.csv": [
       'objID', 'RA_ICRS', 'DE_ICRS', 'Jmag', 'Hmag', 'Kmag', 'e_Jmag', 'e_Hmag', 'e_Kmag', 'umag',
       'gmag', 'rmag', 'imag', 'zmag', 'zsp'
  ],# ------------------------------------------------------------------------------------------
    "STARHORSE2021_SDSSDR16.csv": [
       'objID', 'RA_ICRS', 'DE_ICRS', 'mass50', 'met50', 'AV50', 'dist50', 'zsp'
  ],# ------------------------------------------------------------------------------------------
    "GALAXGR6+7AIS_SDSSDR16.csv": [
       'objID', 'RA_ICRS', 'DE_ICRS', 'FUVmag', 'NUVmag', 'e_FUVmag', 'e_NUVmag', 'zsp'
  ],#----------------------------------------------------------------------------------------
    "UKIDSSDR9LAS_SDSSDR16.csv": [
       'objID', 'RA_ICRS', 'DE_ICRS', 'yAperMag3', 'j_1AperMag3', 'hAperMag3', 'kAperMag3', 'zsp'
  ],# ------------------------------------------------------------------------------------------
    "GAIADR3AP_SDSSDR16.csv": [
       'objID', 'RA_ICRS', 'DE_ICRS', 'A0', 'Age-Flame', 'Dist', 'Teff', 'logg', '[Fe/H]', 'zsp'
  ],# ------------------------------------------------------------------------------------------
    "PanST2DR1_SDSSDR16.csv": [
       'objID', 'RA_ICRS', 'DE_ICRS', 'umag', 'gmag', 'rmag', 'imag', 'zmag', 'e_umag', 'e_gmag',
       'e_rmag', 'e_imag', 'e_zmag', 'zsp'
  ],# ------------------------------------------------------------------------------------------
    "PanSTDR1_SDSSDR16.csv": [
       'objID', 'RA_ICRS', 'DE_ICRS', 'umag', 'gmag', 'rmag', 'imag', 'zmag', 'e_umag', 'e_gmag',
       'e_rmag', 'e_imag', 'e_zmag', 'zsp'
  ],# ------------------------------------------------------------------------------------------
    "SDSS_MangaSpec_with_fluxes.csv": [
```

```
        'SpecObj', 'OH_O3N2_cen', 'OH_T04_cen', 'Av_gas_Re',          'vel_disp_Ha_cen', 'Sigma_Mass_Re',
        'Age_LW_Re_fit', 'ZH_LW_Re_fit', 'EW_Ha_cen', 'stellar_mass', 'Re_kpc',          'redshift',          'flux_Ha',
        'flux_Hb', 'flux_OIII_5007', 'flux_NII_6584', 'e_flux_Ha'
    ],# ----------------------------------------------------------------------------------------------------
    "merged_galspec_gz2.csv": [
        'objid',   'z', 'logmass',     'petrorad_r', 'ellipticity', 'sfr',         'metallicity', 'mRrCc_r', 'ra',         'dec'
    ]
}
# Function to standardize RA/Dec
# ============================================================================================
# ============================================================================
def standardize_columns(df, cols):
    df = df[cols].copy()
    if 'RA_ICRS' in df.columns and 'DE_ICRS' in df.columns:
        df = df.rename(columns={"RA_ICRS": "objra_y", "DE_ICRS": "objdec"})
    elif 'ra' in df.columns and 'dec' in df.columns:
        df = df.rename(columns={"ra": "objra_y", "dec": "objdec"})
    return df

# Incremental merge
# ============================================================================================
# ============================================================================
print("Merging Datasets...")
merged_df = None
for i, file in tqdm(enumerate(csv_files), total=len(csv_files), desc="Merging CSV Files"):
    print(f"Loading {file} ({i+1}/{len(csv_files)})...")
    try:
        chunksize = 50000
        df_chunks = pd.read_csv(file, usecols=column_mappings[file], chunksize=chunksize)
        df = pd.concat([standardize_columns(chunk, column_mappings[file]) for chunk in df_chunks], ignore_index=True)
        print(f"Rows in {file}: {len(df)}, Columns: {len(df.columns)}")
        if 'objid' in df.columns:
            print(f"objid present in {file}")
        else:
            print(f"No objid in {file}")
    except ValueError as e:
        print(f"Error loading {file}: {e}. Check column names.")
        continue

    if merged_df is None:
        merged_df = df
    else:
        merge_keys = ['objra_y', 'objdec']
        if 'objid' in merged_df.columns and 'objid' in df.columns:
            merge_keys.append('objid')
        merged_df = merged_df.merge(df, on=merge_keys, how='left', suffixes=('', f'_dup_{i}'))
        dup_cols = [col for col in merged_df.columns if f'_dup_{i}' in col]
        merged_df = merged_df.drop(columns=dup_cols)
```

```
    merged_df = merged_df.drop_duplicates(subset=['objra_y', 'objdec'], keep='first')
    print(f"Rows after merging {file}: {len(merged_df)}")
# - Save intermediate result per file
-------------------------------------------------------------------------------------------------------------------
    temp_file = f"temp_merge_step_{i+1}.csv"
    merged_df.to_csv(temp_file, index=False)
    print(f"Saved to {temp_file}")

# Final assignment and save after all merges
================================================================================================
================================
df = merged_df
df.to_csv("*STAR_preprocess.csv", index=False)
print(f"Completed merging. Final rows: {len(df)}, columns: {len(df.columns)}. Saved to *STAR_preprocess.csv")
print("Preprocessing completed. Engineering features...")

# Clean metallicity and sfr columns by replacing -9999 with NaN
================================================================================================
==============
df[['metallicity', 'sfr']] = df[['metallicity', 'sfr']].replace(-9999, np.nan)
print(f"Replaced -9999 with NaN in 'metallicity' and 'sfr' columns.")
print(f"NaN in metallicity: {df['metallicity'].isna().sum()}, NaN in sfr: {df['sfr'].isna().sum()}")

# Merge Diagnostics
================================================================================================
==========================================================
key_columns = ['objra_y', 'objdec', 'zsp', 'flux_Ha', 'logmass', 'petrorad_r']
missing_cols = [col for col in key_columns if col not in df.columns]
if missing_cols:
    print(f"Warning: Missing key columns after merge: {missing_cols}")
else:
    print("All key columns present after merge.")
print(f"Sample data:\n{df[key_columns].head() if not missing_cols else df.head()}")

# Downcast numeric columns to save memory
================================================================================================
====================================
def downcast_df(df):
    for col in df.select_dtypes(include=['int']).columns:
        df[col] = pd.to_numeric(df[col], downcast='integer')
    for col in df.select_dtypes(include=['float']).columns:
        df[col] = pd.to_numeric(df[col], downcast='float')
    return df
df = downcast_df(df)
print(f"Memory usage after downcast: {df.memory_usage().sum() / 1024**2:.2f} MB")

# Ensure RA/Dec are numeric and impute NaN/infinities with medians
================================================================================================
===========
```

```
df['objra_y'] = pd.to_numeric(df['objra_y'], errors='coerce')
df['objdec'] = pd.to_numeric(df['objdec'], errors='coerce')
print(f"Initial df: objra_y dtype: {df['objra_y'].dtype}, objdec dtype: {df['objdec'].dtype}")
print(f"NaN in objra_y: {df['objra_y'].isna().sum()}, NaN in objdec: {df['objdec'].isna().sum()}")
df['objra_y'] = df['objra_y'].replace([np.inf, -np.inf], np.nan).fillna(df['objra_y'].median(skipna=True))
df['objdec'] = df['objdec'].replace([np.inf, -np.inf], np.nan).fillna(df['objdec'].median(skipna=True))
print(f"Rows after RA/Dec imputation: {len(df)}")

# Impute NaN/infinities for all numeric columns
================================================================================================
==============================
numeric_cols = df.select_dtypes(include=[np.number]).columns
for col in numeric_cols:
    df[col] = df[col].replace([np.inf, -np.inf], np.nan).fillna(df[col].median(skipna=True))

# Recompute log_SFR_Ha and metallicity
================================================================================================
======================================
flux_cols = ['flux_Ha', 'flux_Hb', 'flux_OIII_5007', 'flux_NII_6584', 'e_flux_Ha']
for col in flux_cols:
    print(f"NaN count in {col} after imputation: {df[col].isna().sum()}")
df['Ha_Hb_observed'] = df['flux_Ha'] / df['flux_Hb']
Ha_Hb_intrinsic = 2.86
k_Ha = 2.468
k_Hb = 3.634
df['A_Ha'] = 2.5 * np.log10(df['Ha_Hb_observed'] / Ha_Hb_intrinsic) * (k_Ha / (k_Hb - k_Ha))
df['flux_Ha_corr'] = df['flux_Ha'] * 10**(0.4 * df['A_Ha'])

# Define cosmology
================================================================================================
==========================================================
cosmo = FlatLambdaCDM(H0=70, Om0=0.3)
sfr_conversion = 7.9e-42 * u.Msun / u.yr / u.erg * u.s  # Kennicutt 1998, Salpeter IMF

# Cluster galaxies using RA/Dec and redshift (optimized with parallelization)
================================================================================================
print("initiating galaxy clustering...")
start_time = time.time()

# Convert coordinates to Cartesian for clustering
================================================================================================
============================
coords = SkyCoord(ra=df['objra_y']*u.deg, dec=df['objdec']*u.deg, distance=df['zsp']*cosmo.hubble_distance,
frame='icrs')
xyz = coords.cartesian.xyz.value.T  # Shape: (n_rows, 3)

# Build the cKDTree
================================================================================================
==========================================================
```

```
tree = cKDTree(xyz)
dist_threshold = 1 / cosmo.hubble_distance.value  # ~1 Mpc

# Function to compute neighbors for a chunk of indices
================================================================================================
======================
def compute_neighbors_chunk(chunk_indices, xyz_data, dist_thr):
    local_tree = cKDTree(xyz_data)  # Rebuild tree for consistency (small overhead)
    return [len(local_tree.query_ball_point(xyz_data[i], r=dist_thr)) - 1 for i in chunk_indices]

# Parallelize the neighbor search
================================================================================================
==========================================
n_cores = mp.cpu_count()  # Use all available cores on SciServer
chunk_size = len(df) // n_cores
if chunk_size == 0:
    chunk_size = len(df)  # Handle case where n_rows < n_cores

index_chunks = [range(i, min(i + chunk_size, len(df))) for i in range(0, len(df), chunk_size)]
pool = mp.Pool(processes=n_cores)
compute_neighbors_partial = partial(compute_neighbors_chunk, xyz_data=xyz, dist_thr=dist_threshold)

# Compute neighbors in parallel
================================================================================================
==========================================
results = pool.map(compute_neighbors_partial, index_chunks)
pool.close()
pool.join()

# Flatten the results
================================================================================================
======================================================
cluster_density = []
for chunk_result in results:
    cluster_density.extend(chunk_result)

df['cluster_density'] = cluster_density

# Diagnostics
================================================================================================
================================================================================
end_time = time.time()
print(f"Clustering completed in {end_time - start_time:.2f} seconds.")
print(f"Memory usage after clustering: {df.memory_usage().sum() / 1024**2:.2f} MB")
print(f"cluster_density: mean={df['cluster_density'].mean():.2f}, std={df['cluster_density'].std():.2f}, "
      f"min={df['cluster_density'].min():.2f}, max={df['cluster_density'].max():.2f}")

# Galactic extinction correction (enhanced with caching and robust error handling)
================================================================================
```

```
print("Applying galactic extinction correction...")
start_time = time.time()

# Define cache file for ebv values
================================================================================================
=========================================
ebv_cache_file = "ebv_cache.pkl"

# Check if cached ebv values exist
================================================================================================
=========================================
if os.path.exists(ebv_cache_file):
    print(f"Loading cached EBV values from {ebv_cache_file}...")
    with open(ebv_cache_file, 'rb') as f:
        ebv = pickle.load(f)
else:
# = Initialize SFDQuery
================================================================================================
=====================================================
    try:
        sfd = SFDQuery()
    except FileNotFoundError:
        print("SFD dustmap not found. Fetching now...")
        try:
            import dustmaps.sfd
            dustmaps.sfd.fetch()
            sfd = SFDQuery()
        except Exception as e:
            print(f"Failed to fetch SFD dustmap due to {e}. Will use fallback.")
            sfd = None
    except Exception as e:
        print(f"Failed to initialize SFDQuery due to {e}. Will use fallback.")
        sfd = None
# = Compute ebv values ----------------------------------------------------------------------------------------
    if sfd is not None:
        try:
# ========= Validate coordinates ------------------------------------------------------------------------------
            if not coords.is_finite().all():
                raise ValueError("Invalid coordinates detected in SkyCoord object.")
            ebv = sfd(coords)
            ebv = np.array(ebv, dtype=np.float32)
# ======== Validate ebv values --------------------------------------------------------------------------------
            ebv = np.where(np.isfinite(ebv) & (ebv >= 0), ebv, 0.0)  # Replace NaN/inf and negative values with 0
# ======== Cache the ebv values -------------------------------------------------------------------------------
            with open(ebv_cache_file, 'wb') as f:
                pickle.dump(ebv, f)
            print(f"Cached EBV values to {ebv_cache_file}.")
        except Exception as e:
            print(f"Failed to compute EBV due to {e}. Will use fallback.")
```

```
            ebv = None
    else:
        ebv = None
# Fallback if ebv computation failed ----------------------------------------------------------------------------
if ebv is None:
    print("Warning: Using median-based fallback for galactic extinction correction.")
# = Estimate a reasonable A_Ha_mw based on typical EBV values (e.g., 0.05 as a median for low-latitude regions)
=======================================================
    typical_ebv = 0.05
    A_Ha_mw = k_Ha * typical_ebv * 3.1  # Rv = 3.1
    A_Ha_mw = np.full(len(df), A_Ha_mw, dtype=np.float32)
else:
    A_Ha_mw = k_Ha * ebv * 3.1  # Rv = 3.1
# Diagnostics for A_Ha_mw ---------------------------------------------------------------------------------------
print(f"A_Ha_mw: mean={np.mean(A_Ha_mw):.4f}, std={np.std(A_Ha_mw):.4f}, "
      f"min={np.min(A_Ha_mw):.4f}, max={np.max(A_Ha_mw):.4f}")
if ebv is not None:
    print(f"EBV: mean={np.mean(ebv):.4f}, std={np.std(ebv):.4f}, "
          f"min={np.min(ebv):.4f}, max={np.max(ebv):.4f}")

# Update flux_Ha_corr with extinction correction
=============================================================================================
============================
df['flux_Ha_corr'] = df['flux_Ha'] * 10**(0.4 * (df['A_Ha'] + A_Ha_mw))

# Diagnostics for flux_Ha_corr
=============================================================================================
=============================================
print(f"flux_Ha_corr: mean={df['flux_Ha_corr'].mean():.4f}, std={df['flux_Ha_corr'].std():.4f}, "
      f"NaN={df['flux_Ha_corr'].isna().sum()}, Inf={np.isinf(df['flux_Ha_corr']).sum()}")

end_time = time.time()
print(f"Galactic extinction correction completed in {end_time - start_time:.2f} seconds.")

# Compute luminosity distance and SFR
=============================================================================================
=======================================
df['DL'] = cosmo.luminosity_distance(df['zsp']).to(u.cm).value
df['L_Ha'] = (df['flux_Ha_corr'] * 4 * np.pi * df['DL']**2 * u.erg / u.s).to(u.L_sun).value
df['log_SFR_Ha_raw'] = np.log10((df['L_Ha'] * sfr_conversion).value)
print(f"NaN in L_Ha: {df['L_Ha'].isna().sum()}")
print(f"NaN in log_SFR_Ha_raw: {df['log_SFR_Ha_raw'].isna().sum()}")

# Add raw fluxes/morpholohicals as features
=============================================================================================
================================
df['log_flux_Ha'] = np.log10(df['flux_Ha'] + 1e-10)
df['log_flux_Hb'] = np.log10(df['flux_Hb'] + 1e-10)
df['log_flux_OIII_5007'] = np.log10(df['flux_OIII_5007'] + 1e-10)
```

```
df['log_flux_NII_6584'] = np.log10(df['flux_NII_6584'] + 1e-10)
df['log_e_flux_Ha'] = np.log10(df['e_flux_Ha'] + 1e-10)

# Compute metallicity
================================================================================================
======================================================
df['log_O3N2_raw'] = np.log10((df['flux_OIII_5007'] / df['flux_Hb']) / (df['flux_NII_6584'] / df['flux_Ha']) + 1e-10)
df['OH_O3N2_raw'] = 8.533 - 0.214 * df['log_O3N2_raw']

# Add photometric colors
================================================================================================
======================================================
if 'umag' in df.columns and 'gmag' in df.columns:
    df['color_ug'] = df['umag'] - df['gmag']
    df['color_gr'] = df['gmag'] - df['rmag']
    df['color_ri'] = df['rmag'] - df['imag']
    df['color_iz'] = df['imag'] - df['zmag']
    df['e_color_ug'] = np.sqrt(df['e_umag']**2 + df['e_gmag']**2)
    df['e_color_gr'] = np.sqrt(df['e_gmag']**2 + df['e_rmag']**2)
if 'Jmag' in df.columns and 'Kmag' in df.columns:
    df['color_JK'] = df['Jmag'] - df['Kmag']
    df['e_color_JK'] = np.sqrt(df['e_Jmag']**2 + df['e_Kmag']**2)

# Step 1: Cosmo-Rank Construction (flux or morphological features)
================================================================================================
===========
flux_cols = ['flux_Ha', 'flux_Hb', 'flux_OIII_5007', 'flux_NII_6584', 'e_flux_Ha']
if 'ellipticity' in df.columns and ('zsp' in df.columns or 'z' in df.columns):
    gz_df = df[['ellipticity']].copy()
    gz_df['redshift'] = df['zsp'] if 'zsp' in df.columns else df['z']
    gz_df['morph_proxy'] = gz_df['ellipticity']
    gz_df['Num_w'] = 1.0
    scaler_rank = MinMaxScaler()
    gz_df[['morph_norm', 'Num_w_norm', 'redshift_norm']] = scaler_rank.fit_transform(
        gz_df[['morph_proxy', 'Num_w', 'redshift']]
    )
    df['cosmo_rank'] = 0.5 * gz_df['Num_w_norm'] + 0.3 * gz_df['morph_norm'] + 0.2 * gz_df['redshift_norm']
    print("Computed cosmo_rank using ellipticity and redshift proxies.")
else:
    print("Warning: ellipticity or redshift missing. Setting cosmo_rank to 0.5.")
    df['cosmo_rank'] = 0.5

# Ensure cosmo_rank is numeric and imputed
================================================================================================
==================================
df['cosmo_rank'] = pd.to_numeric(df['cosmo_rank'], errors='coerce').replace([np.inf, -np.inf], np.nan).fillna(0.5)
```

```
# Step 2: L_cosmo(s) Construction
================================================================================================
==========================================
if 'logmass' in df.columns:
    df['logMass'] = df['logmass']
if 'petrorad_r' in df.columns:
    df['kronRad'] = df['petrorad_r']

alpha = -1.5
if 'logMass' in df.columns and 'kronRad' in df.columns:
    df['log_Mass_gas'] = df['logMass']
    M_star = df['log_Mass_gas'].median()
    df['a_n'] = (10 ** df['log_Mass_gas'])**(1 + alpha) * np.exp(-10 ** df['log_Mass_gas'] / (10 ** M_star))

# = Preprocess kronRad ----------------------------------------------------------------------------------
    df['kronRad'] = df['kronRad'].replace([np.inf, -np.inf], np.nan).fillna(df['kronRad'].median(skipna=True))
    if df['kronRad'].isna().all():
        print("Warning: kronRad is all NaN after preprocessing. Setting L_cosmo_s_* to 0.")
        for s in [0.5, 1.0, 1.5, 2.0]:
            df[f'L_cosmo_s_{s:.1f}'] = 0
    else:

# ======= Try quantile binning first ---------------------------------------------------------------------
        try:
            bins = pd.qcut(df['kronRad'], q=20, labels=False, duplicates='drop') + 1
            df['z_bin'] = bins
            n_bins = df['z_bin'].nunique()
            print(f"Number of unique bins (qcut): {n_bins}")
            print("Bin distribution for kronRad (qcut):")
            print(df['z_bin'].value_counts().sort_index())
        except Exception as e:
            print(f"Warning: pd.qcut failed due to {e}. Falling back to pd.cut.")

# ======== Fallback to equal-width binning ----------------------------------------------------------------
            try:
                bins = pd.cut(df['kronRad'], bins=20, labels=False, duplicates='drop') + 1
                df['z_bin'] = bins
                n_bins = df['z_bin'].nunique()
                print(f"Number of unique bins (cut): {n_bins}")
                print("Bin distribution for kronRad (cut):")
                print(df['z_bin'].value_counts().sort_index())
            except Exception as e:
                print(f"Warning: pd.cut also failed due to {e}. Using median-based fallback.")
                df['z_bin'] = (df['kronRad'] > df['kronRad'].median()).astype(int) + 1  # Binary split
                n_bins = df['z_bin'].nunique()
                print(f"Number of unique bins (fallback): {n_bins}")
                print("Bin distribution for kronRad (fallback):")
                print(df['z_bin'].value_counts().sort_index())
```

```
# ======== Ensure a minimum number of bins ----------------------------------------------------------------------------
        if n_bins < 5:
            print(f"Warning: Only {n_bins} unique bins. Using fallback binary split to ensure variation.")
            df['z_bin'] = (df['kronRad'] > df['kronRad'].median()).astype(int) + 1  # Binary split
            n_bins = df['z_bin'].nunique()
            print(f"Adjusted number of unique bins: {n_bins}")
            print("Adjusted bin distribution for kronRad:")
            print(df['z_bin'].value_counts().sort_index())

# ======== Compute L_cosmo_s_* features -------------------------------------------------------------------------------
        s_vals = [0.5, 1.0, 1.5, 2.0]
        for s in s_vals:
            col_name = f'L_cosmo_s_{s:.1f}'
            df[col_name] = df.groupby('z_bin')['a_n'].transform('mean') / (df['z_bin'] ** s)

# ======== Replace any NaN or inf values in L_cosmo_s_* with 0 --------------------------------------------------------
            df[col_name] = df[col_name].replace([np.inf, -np.inf], np.nan).fillna(0)
            print(f"Created {col_name}")
else:
    print("Warning: logMass or kronRad missing. Setting L_cosmo_s_* to 0.")
    for s in [0.5, 1.0, 1.5, 2.0]:
        df[f'L_cosmo_s_{s:.1f}'] = 0

# Diagnostics for L_cosmo_s_* features
================================================================================================
=====================================
print(f"cosmo_rank: mean={df['cosmo_rank'].mean():.4f}, std={df['cosmo_rank'].std():.4f}")
for s in [0.5, 1.0, 1.5, 2.0]:
    col = f'L_cosmo_s_{s:.1f}'
    print(f"{col}: mean={df[col].mean():.4f}, std={df[col].std():.4f}, NaN={df[col].isna().sum()},
Inf={np.isinf(df[col]).sum()}")

# Feature engineering with proper cosmic measurements
================================================================================================
========================
df['log_Mass'] = df['log_Mass_gas']

# Check for missing columns and compute if necessary
================================================================================================
========================
required_cols = {
    'SpecObj': 'SpecObj',
    'OH_O3N2_cen': 'OH_O3N2_cen',
    'OH_T04_cen': 'OH_T04_cen',
    'Av_gas_Re': 'Av_gas_Re',
    'vel_disp_Ha_cen': 'vel_disp_Ha_cen',
    'Sigma_Mass_Re': 'Sigma_Mass_Re',
    'Age_LW_Re_fit': 'Age_LW_Re_fit',
    'ZH_LW_Re_fit': 'ZH_LW_Re_fit',
```

```
    'EW_Ha_cen': 'EW_Ha_cen',
    'stellar_mass': 'stellar_mass',
    'Re_kpc': 'Re_kpc',
    'redshift': 'redshift',
    'cosmo_rank_L': 'cosmo_rank'
}
for col, fallback in required_cols.items():
    if col not in df.columns:
        if col == 'Av_gas_Re':
            if 'AV50' in df.columns and 'A0' in df.columns:
                df['Av_gas_Re'] = df['AV50'].fillna(df['A0']).fillna(0)
            elif 'AV50' in df.columns:
                df['Av_gas_Re'] = df['AV50'].fillna(0)
            elif 'A0' in df.columns:
                df['Av_gas_Re'] = df['A0'].fillna(0)
            else:
                df['Av_gas_Re'] = 0
            print("Assigned Av_gas_Re from AV50/A0")
        elif col == 'Age_LW_Re_fit':
            df['Age_LW_Re_fit'] = df.get('Age-Flame', 0)
            print("Assigned Age_LW_Re_fit from Age-Flame")
        elif col == 'ZH_LW_Re_fit':
            df['ZH_LW_Re_fit'] = df.get('met50', 0)
            print("Assigned ZH_LW_Re_fit from met50")
        elif col == 'cosmo_rank_L':
            df['cosmo_rank_L'] = df.get('cosmo_rank', 0.5)
            print("Assigned cosmo_rank_L from cosmo_rank")
        elif fallback:
            df[col] = df.get(fallback, np.nan)
            print(f"Warning: {col} not found. Using {fallback} as fallback.")
        else:
            print(f"Warning: {col} not found. Setting to 0.")
            df[col] = 0

# Feature engineering block with cosmic adjustments
============================================================================================
==========================
print(f"Memory usage before feature engineering: {df.memory_usage().sum() / 1024**2:.2f} MB")

# Precompute constants and reusable series to avoid redundant calculations
============================================================================================
===
logMass = df['logMass'].values  # Convert to NumPy array for efficiency
cosmo_rank = df['cosmo_rank'].values
L_cosmo_s1 = df['L_cosmo_s_1.0'].values

# Dictionary to store new features as NumPy arrays
============================================================================================
==========================
```

```
new_features_dict = {}

# Compute new features using NumPy for efficiency
================================================================================================
============================
new_features_dict['mass_metallicity'] = logMass * df['OH_O3N2_raw'].values
new_features_dict['dust_metallicity'] = df['Av_gas_Re'].values * df.get('OH_T04_cen', 0).values
new_features_dict['disp_mass_ratio'] = df['vel_disp_Ha_cen'].values / (df['Sigma_Mass_Re'].values + 1e-5)
new_features_dict['age_metallicity'] = df['Age_LW_Re_fit'].values * df['ZH_LW_Re_fit'].values
new_features_dict['sqrt_Re_kpc'] = np.sqrt(df['Re_kpc'].values + 1e-5)
new_features_dict['BSD_likelihood'] = logMass * df['OH_O3N2_raw'].values / (df['Av_gas_Re'].values +
df['Age_LW_Re_fit'].values + 1e-5)
new_features_dict['cosmo_rank_mass'] = cosmo_rank * logMass
new_features_dict['L_cosmo_s1_mass'] = L_cosmo_s1 * logMass
new_features_dict['cosmo_rank_EW'] = cosmo_rank * df['EW_Ha_cen'].values
new_features_dict['L_cosmo_s1_EW'] = L_cosmo_s1 * df['EW_Ha_cen'].values
new_features_dict['cosmo_rank_scaled'] = cosmo_rank * 20
new_features_dict['L_cosmo_s1_scaled'] = L_cosmo_s1 * 20
new_features_dict['L_cosmo_s1_metallicity'] = L_cosmo_s1 * df['OH_O3N2_raw'].values

# Impute new features one at a time to reduce memory overhead
================================================================================================
================
new_features = [
    "mass_metallicity", "dust_metallicity", "disp_mass_ratio", "age_metallicity",
    "sqrt_Re_kpc", "BSD_likelihood", "cosmo_rank_mass", "L_cosmo_s1_mass",
    "cosmo_rank_EW", "L_cosmo_s1_EW", "cosmo_rank_scaled", "L_cosmo_s1_scaled",
    "L_cosmo_s1_metallicity"
]

for feature in new_features:
    # Replace inf with NaN and compute median ----------------------------------------------------
    arr = np.where(np.isfinite(new_features_dict[feature]), new_features_dict[feature], np.nan)
    median_val = np.nanmedian(arr)
    # Impute NaN with median ---------------------------------------------------------------------
    new_features_dict[feature] = np.where(np.isnan(arr), median_val, arr)
    # Assign to DataFrame ------------------------------------------------------------------------
    df[feature] = new_features_dict[feature]
    del new_features_dict[feature]

# Free memory of precomputed arrays ----------------------------------------------------------
del logMass, cosmo_rank, L_cosmo_s1
print(f"Memory usage after feature engineering: {df.memory_usage().sum() / 1024**2:.2f} MB")

# Define final features list
================================================================================================
================================================
target = "log_SFR_Ha_raw"
features = [
```

```
    "logMass", "Re_kpc", "OH_O3N2_raw", "cosmo_rank",
    "mass_metallicity", "sqrt_Re_kpc", "cosmo_rank_mass", "L_cosmo_s1_mass",
    "log_flux_Ha", "log_flux_Hb", "log_flux_OIII_5007", "log_flux_NII_6584",
    "log_e_flux_Ha", "L_cosmo_s_0.5", "L_cosmo_s_1.0", "L_cosmo_s_1.5",
    "L_cosmo_s_2.0", "cluster_density"
]
if all(col in df.columns for col in ['umag', 'gmag', 'rmag', 'imag', 'zmag']):
    features.extend(["color_ug", "color_gr", "color_ri", "color_iz", "e_color_ug", "e_color_gr"])
if all(col in df.columns for col in ['Jmag', 'Kmag']):
    features.extend(["color_JK", "e_color_JK"])
df[new_features] = df[new_features].replace([np.inf, -np.inf], np.nan).fillna(df[new_features].median(skipna=True))

# BSD_likelihood diagnostics
================================================================================================
================================================
print("\nBSD_likelihood Diagnostics:")
print(f"NaN count: {df['BSD_likelihood'].isna().sum()}")
print(f"Mean: {df['BSD_likelihood'].mean():.4f}")
print(f"Median: {df['BSD_likelihood'].median():.4f}")
print(f"Std: {df['BSD_likelihood'].std():.4f}")
print(f"Min: {df['BSD_likelihood'].min():.4f}, Max: {df['BSD_likelihood'].max():.4f}")

plt.figure(figsize=(8, 6))
plt.hist(df['BSD_likelihood'], bins=50, range=(df['BSD_likelihood'].quantile(0.01), df['BSD_likelihood'].quantile(0.99)),
density=True, alpha=0.7)
plt.xlabel("BSD_likelihood")
plt.ylabel("Density")
plt.title("Distribution of BSD_likelihood")
plt.savefig("BSD_likelihood_histogram.png", dpi=150)
plt.close()

if 'log_SFR_Ha_raw' in df.columns:
    mask = df['BSD_likelihood'].notna() & df['log_SFR_Ha_raw'].notna()
    plt.figure(figsize=(8, 6))
    plt.scatter(df.loc[mask, 'BSD_likelihood'], df.loc[mask, 'log_SFR_Ha_raw'], alpha=0.5, s=10)
    plt.xlabel("BSD_likelihood")
    plt.ylabel("log_SFR_Ha_raw")
    plt.title("BSD_likelihood vs log_SFR_Ha_raw")
    plt.savefig("BSD_likelihood_vs_SFR.png", dpi=150)
    plt.close()

# Verify feature availability
================================================================================================
================================================================
missing_features = [f for f in features if f not in df.columns]
if missing_features:
    print(f"Warning: Missing features: {missing_features}")
```

```
# Save final dataset
================================================================================
================================================================================
df.to_csv("*STAR_dataset.csv", index=False)
print("Saved to *STAR_dataset.csv")

# Post-processing diagnostics
================================================================================
================================================================================
key_features = [
    'log_SFR_Ha_raw', 'log_Mass_gas', 'Re_kpc', 'flux_Ha', 'cosmo_rank', 'cluster_density', 'OH_O3N2_raw',
    'L_cosmo_s_0.5', 'L_cosmo_s_1.0', 'L_cosmo_s_1.5', 'L_cosmo_s_2.0',  # Add L_cosmo_s_* features
    'BSD_likelihood', 'mass_metallicity', 'cosmo_rank_mass', 'L_cosmo_s1_mass',  # Add engineered features
    'log_flux_Ha', 'log_flux_Hb', 'log_flux_OIII_5007', 'log_flux_NII_6584', 'log_e_flux_Ha'  # Add flux/morphology
features
]
print("\nPost-processing diagnostics:")
for col in key_features:
    if col in df.columns:
        nan_count = df[col].isna().sum()
        inf_count = np.isinf(df[col]).sum()
        mean_val = df[col].mean()
        median_val = df[col].median()
        std_val = df[col].std()
        min_val = df[col].min()
        max_val = df[col].max()
        sample_vals = df[col].iloc[:5].tolist()
        print(f"{col}: NaN = {nan_count}, Inf = {inf_count}, Mean = {mean_val:.4f}, Median = {median_val:.4f}, "
              f"Std = {std_val:.4f}, Min = {min_val:.4f}, Max = {max_val:.4f}, Sample = {sample_vals}")

#===== Plot histogram for each feature
================================================================================
============================================================
        plt.figure(figsize=(8, 6))
        plt.hist(df[col].dropna(), bins=50, range=(df[col].quantile(0.01), df[col].quantile(0.99)),
                 density=True, alpha=0.7)
        plt.xlabel(col)
        plt.ylabel("Density")
        plt.title(f"Distribution of {col}")
        plt.savefig(f"dist_{col}.png", dpi=150)
        plt.close()
    else:
        print(f"{col}: Missing from dataset")

# Define s_range and plot L_cosmo_s vs log_SFR_Ha_raw
================================================================================
==========================================
s_vals = [0.5, 1.0, 1.5, 2.0]
for s in s_vals:
```

```
    col_name = f'L_cosmo_s_{s:.1f}'
    if col_name in df.columns:
        mask = df[col_name].notna() & df['log_SFR_Ha_raw'].notna() & np.isfinite(df[col_name]) &
np.isfinite(df['log_SFR_Ha_raw'])
        if mask.sum() < 10:
            print(f"Warning: Insufficient valid data for {col_name} plotting/fitting ({mask.sum()} valid rows).")
            continue

# ===== Scatter plot
================================================================================================
================================================================================
        plt.figure(figsize=(8, 6))
        plt.scatter(df.loc[mask, col_name], df.loc[mask, 'log_SFR_Ha_raw'], alpha=0.5, s=10)
        plt.xlabel(f"{col_name}")
        plt.ylabel("log_SFR_Ha_raw")
        plt.title(f"{col_name} vs log_SFR_Ha_raw")
        plt.savefig(f"L_cosmo_s{s:.1f}_vs_SFR.png", dpi=150)
        plt.close()

# ==== Polynomial fit plot
================================================================================================
================================================================================
        try:
            poly_coeffs = Polynomial.fit(df.loc[mask, col_name], df.loc[mask, 'log_SFR_Ha_raw'], deg=2)
            x_fit = np.linspace(df.loc[mask, col_name].min(), df.loc[mask, col_name].max(), 100)
            y_fit = poly_coeffs(x_fit)
            y_pred = poly_coeffs(df.loc[mask, col_name])
            r2 = r2_score(df.loc[mask, 'log_SFR_Ha_raw'], y_pred)
            plt.figure(figsize=(8, 6))
            plt.scatter(df.loc[mask, col_name], df.loc[mask, 'log_SFR_Ha_raw'], alpha=0.5, s=10)
            plt.plot(x_fit, y_fit, 'r-', label=f'Quadratic Fit (R²={r2:.3f})')
            plt.xlabel(f"{col_name}")
            plt.ylabel("log_SFR_Ha_raw")
            plt.title(f"{col_name} vs log_SFR_Ha_raw with Polynomial Fit")
            plt.legend()
            plt.savefig(f"L_cosmo_s{s:.1f}_vs_SFR_fit.png", dpi=150)
            plt.close()
        except Exception as e:
            print(f"Warning: Polynomial fit failed for {col_name}: {e}")
    else:
        print(f"Warning: {col_name} not available for plotting.")

# Define features and target
================================================================================================
================================================================================
target = "log_SFR_Ha_raw"
features = [
    "logMass", "Re_kpc", "OH_O3N2_raw", "cosmo_rank",
    "mass_metallicity", "sqrt_Re_kpc", "cosmo_rank_mass", "L_cosmo_s1_mass",
```

```
    "log_flux_Ha", "log_flux_Hb", "log_flux_OIII_5007", "log_flux_NII_6584",
    "log_e_flux_Ha", "L_cosmo_s_0.5", "L_cosmo_s_1.0", "L_cosmo_s_1.5",
    "L_cosmo_s_2.0", "cluster_density"
]
if 'umag' in df.columns:
    features.extend(["color_ug", "color_gr", "color_ri", "color_iz", "e_color_ug", "e_color_gr"])
if 'Jmag' in df.columns:
    features.extend(["color_JK", "e_color_JK"])
if 'color_ug' in df.columns:
    features.extend(["color_ug", "color_gr", "color_ri", "color_iz", "e_color_ug", "e_color_gr"])
if 'color_JK' in df.columns:
    features.extend(["color_JK", "e_color_JK"])

# Global imputation before feature selection
================================================================================================
================================================
numeric_cols = df.select_dtypes(include=[np.number]).columns
df[numeric_cols] = df[numeric_cols].replace([np.inf, -np.inf], np.nan).fillna(df[numeric_cols].median(skipna=True))

# Select features and target
================================================================================================
================================================================
X = df[features]
y = df[target]
print(f"Rows after feature selection: {len(X)}")
print(f"NaN in y ({target}): {y.isna().sum()}")
missing_features = [f for f in features if f not in df.columns]
if missing_features:
    print(f"Warning: The following features are missing from the dataset: {missing_features}")
else:
    print("All features present in the dataset.")

# Correlation heatmap of numeric features
================================================================================================
================================================
plt.figure(figsize=(12, 10))
sns.heatmap(X.corr(), annot=False, cmap='coolwarm', vmin=-1, vmax=1)
plt.title("Correlation Heatmap of Features")
plt.savefig("feature_correlation_heatmap.png")
plt.close()

# Filter to numeric columns only
================================================================================================
============================================================
numeric_features = [col for col in features if pd.api.types.is_numeric_dtype(X[col])]
X = X[numeric_features]
print(f"Numeric features selected: {numeric_features}")
print("Sample values:")
print(X.head())
```

```
# Clean X and y with global median imputation
================================================================================================
================================================
X = X.replace([np.inf, -np.inf], np.nan).fillna(X.median(skipna=True))
y = y.replace([np.inf, -np.inf], np.nan).fillna(y.median(skipna=True))
print(f"Rows after X and y imputation: {len(X)}")

# Train/test split
================================================================================================
================================================================================
if len(X) < 10:
    raise ValueError(f"Only {len(X)} rows after cleaning. Check data preprocessing.")
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Verify inputs
================================================================================================
================================================================================
print("NaN in X_train_scaled:", np.isnan(X_train_scaled).sum())
print("Inf in X_train_scaled:", np.isinf(X_train_scaled).sum())
print("NaN in y_train:", y_train.isna().sum())
print("Inf in y_train:", np.isinf(y_train).sum())
print("Feature selection and scaling complete. Initiating *S.T.A.R. training with Optuna...")

# Optuna Optimizations
================================================================================================
================================================================================
def objective_hgb(trial):
    params = {
        'max_iter': trial.suggest_int('max_iter', 100, 500),
        'max_depth': trial.suggest_int('max_depth', 3, 10),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.2, log=True),
        'l2_regularization': trial.suggest_float('l2_regularization', 0.1, 10.0),
        'max_leaf_nodes': trial.suggest_int('max_leaf_nodes', 20, 50),
        'min_samples_leaf': trial.suggest_int('min_samples_leaf', 10, 30),
        'random_state': 42
    }
    model = HistGradientBoostingRegressor(**params)
    cv = KFold(n_splits=5, shuffle=True, random_state=42)
    scores = cross_val_score(model, X_train_scaled, y_train, cv=cv, scoring='r2')
    mean_score = np.nanmean(scores)
    if np.isnan(mean_score):
        print(f"HGB Trial {trial.number} failed: scores = {scores}")
    return mean_score if not np.isnan(mean_score) else -np.inf

def objective_lgb(trial):
```

```
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 500),
        'max_depth': trial.suggest_int('max_depth', 3, 10),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.2, log=True),
        'num_leaves': trial.suggest_int('num_leaves', 20, 50),
        'subsample': trial.suggest_float('subsample', 0.6, 1.0),
        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.6, 1.0),
        'reg_lambda': trial.suggest_float('reg_lambda', 0.1, 10.0),
        'random_state': 42,
        'verbose': -1
    }
    model = lgb.LGBMRegressor(**params)
    cv = KFold(n_splits=5, shuffle=True, random_state=42)
    scores = cross_val_score(model, X_train_scaled, y_train, cv=cv, scoring='r2')
    mean_score = np.nanmean(scores)
    if np.isnan(mean_score):
        print(f"LGB Trial {trial.number} failed: scores = {scores}")
    return mean_score if not np.isnan(mean_score) else -np.inf

def objective_xgb(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 500),
        'max_depth': trial.suggest_int('max_depth', 3, 10),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.2, log=True),
        'subsample': trial.suggest_float('subsample', 0.6, 1.0),
        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.6, 1.0),
        'reg_lambda': trial.suggest_float('reg_lambda', 0.1, 10.0),
        'random_state': 42
    }
    model = xgb.XGBRegressor(**params)
    cv = KFold(n_splits=5, shuffle=True, random_state=42)
    scores = cross_val_score(model, X_train_scaled, y_train, cv=cv, scoring='r2')
    mean_score = np.nanmean(scores)
    if np.isnan(mean_score):
        print(f"XGB Trial {trial.number} failed: scores = {scores}")
    return mean_score if not np.isnan(mean_score) else -np.inf

def objective_cat(trial):
    params = {
        'iterations': trial.suggest_int('iterations', 100, 500),
        'depth': trial.suggest_int('depth', 4, 10),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.2, log=True),
        'l2_leaf_reg': trial.suggest_float('l2_leaf_reg', 1, 10),
        'random_seed': 42,
        'verbose': 0
    }
    model = CatBoostRegressor(**params)
    cv = KFold(n_splits=5, shuffle=True, random_state=42)
    scores = cross_val_score(model, X_train_scaled, y_train, cv=cv, scoring='r2')
```

```
    mean_score = np.nanmean(scores)
    if np.isnan(mean_score):
        print(f"Cat Trial {trial.number} failed: scores = {scores}")
    return mean_score if not np.isnan(mean_score) else -np.inf

# Define models and studies
================================================================================================
================================================================
hgb = HistGradientBoostingRegressor(random_state=42, n_jobs=2)
symbolic_reg = SymbolicRegressor(population_size=1000, generations=20, random_state=42, n_jobs=2)
lgb_model = lgb.LGBMRegressor(random_state=42, verbose=-1)
xgb_model = xgb.XGBRegressor(random_state=42)
cat_model = CatBoostRegressor(random_seed=42, verbose=0)

study_hgb = optuna.create_study(direction='maximize')
study_lgb = optuna.create_study(direction='maximize')
study_xgb = optuna.create_study(direction='maximize')
study_cat = optuna.create_study(direction='maximize')

print("Initiating *S.T.A.R. calculations...")
for model_name in tqdm(['HistGradientBoosting', 'Symbolic', 'LightGBM', 'XGBoost', 'CatBoost'], desc="Training
Models"):
    print(f"Training {model_name}...")
    if model_name == 'HistGradientBoosting':
        study_hgb.optimize(objective_hgb, n_trials=20)
        print(f"Best HistGradientBoosting Parameters: {study_hgb.best_params}")
        print(f"Best HistGradientBoosting CV R²: {study_hgb.best_value:.4f}")
        hgb.set_params(**study_hgb.best_params)
        hgb.fit(X_train_scaled, y_train)
        y_pred_hgb = hgb.predict(X_test_scaled)
        r2_hgb = r2_score(y_test, y_pred_hgb)
        cv_scores_hgb = cross_val_score(hgb, X_train_scaled, y_train, cv=5, scoring='r2')
        print(f"HistGradientBoosting 5-Fold CV R²: Mean = {cv_scores_hgb.mean():.4f}, Std = {cv_scores_hgb.std():.4f}")
        print(f"R^2 Score (HistGradientBoosting, Test): {r2_hgb:.4f}")
    elif model_name == 'Symbolic':
        symbolic_reg.fit(X_train_scaled, y_train)
        y_pred_symbolic = symbolic_reg.predict(X_test_scaled)
        r2_symbolic = r2_score(y_test, y_pred_symbolic)
        print(f"R^2 Score (Symbolic Regression, Test): {r2_symbolic:.4f}")
        print(f"Symbolic Expression: {symbolic_reg._program}")
    elif model_name == 'LightGBM':
        study_lgb.optimize(objective_lgb, n_trials=20)
        print(f"Best LightGBM Parameters: {study_lgb.best_params}")
        print(f"Best LightGBM CV R²: {study_lgb.best_value:.4f}")
        lgb_model.set_params(**study_lgb.best_params)
        lgb_model.fit(X_train_scaled, y_train)
        y_pred_lgb = lgb_model.predict(X_test_scaled)
        r2_lgb = r2_score(y_test, y_pred_lgb)
        cv_scores_lgb = cross_val_score(lgb_model, X_train_scaled, y_train, cv=5, scoring='r2')
```

```
        print(f"LightGBM 5-Fold CV R²: Mean = {cv_scores_lgb.mean():.4f}, Std = {cv_scores_lgb.std():.4f}")
        print(f"R^2 Score (LightGBM, Test): {r2_lgb:.4f}")
    elif model_name == 'XGBoost':
        study_xgb.optimize(objective_xgb, n_trials=20)
        print(f"Best XGBoost Parameters: {study_xgb.best_params}")
        print(f"Best XGBoost CV R²: {study_xgb.best_value:.4f}")
        xgb_model.set_params(**study_xgb.best_params)
        xgb_model.fit(X_train_scaled, y_train)
        y_pred_xgb = xgb_model.predict(X_test_scaled)
        r2_xgb = r2_score(y_test, y_pred_xgb)
        cv_scores_xgb = cross_val_score(xgb_model, X_train_scaled, y_train, cv=5, scoring='r2')
        print(f"XGBoost 5-Fold CV R²: Mean = {cv_scores_xgb.mean():.4f}, Std = {cv_scores_xgb.std():.4f}")
        print(f"R^2 Score (XGBoost, Test): {r2_xgb:.4f}")
    elif model_name == 'CatBoost':
        study_cat.optimize(objective_cat, n_trials=20)
        print(f"Best CatBoost Parameters: {study_cat.best_params}")
        print(f"Best CatBoost CV R²: {study_cat.best_value:.4f}")
        cat_model.set_params(**study_cat.best_params)
        cat_model.fit(X_train_scaled, y_train)
        y_pred_cat = cat_model.predict(X_test_scaled)
        r2_cat = r2_score(y_test, y_pred_cat)
        cv_scores_cat = cross_val_score(cat_model, X_train_scaled, y_train, cv=5, scoring='r2')
        print(f"CatBoost 5-Fold CV R²: Mean = {cv_scores_cat.mean():.4f}, Std = {cv_scores_cat.std():.4f}")
        print(f"R^2 Score (CatBoost, Test): {r2_cat:.4f}")

# Blend symbolic regression with HistGradientBoosting
================================================================================================
=========================================
y_pred_blend = (y_pred_hgb + y_pred_symbolic) / 2
r2_blend = r2_score(y_test, y_pred_blend)
print(f"R^2 Score (Blended HGB + Symbolic, Test): {r2_blend:.4f}")
y_pred_all = (hgb.predict(X_train_scaled) + symbolic_reg.predict(X_train_scaled)) / 2
percentiles = np.percentile(y_pred_all - y_train, [5, 95])
print(f"Blended Model 90% Prediction Interval: [{percentiles[0]:.4f}, {percentiles[1]:.4f}]")

# Plot blended predictions vs actual
================================================================================================
=========================================================
plt.scatter(y_test, y_pred_blend, alpha=0.5)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.xlabel("Actual log_SFR_Ha_raw")
plt.ylabel("Predicted log_SFR_Ha_raw (Blended HGB + Symbolic)")
plt.title("Blended Model: Actual vs Predicted")
plt.savefig("blended_actual_vs_pred.png", dpi=150)
plt.close()

# SHAP Analysis
================================================================================================
=============================================================================
```

```
sample_idx = np.random.choice(X_test_scaled.shape[0], min(1000, X_test_scaled.shape[0]), replace=False)
X_test_scaled_sample = X_test_scaled[sample_idx]

explainer_hgb = shap.Explainer(hgb, X_train_scaled)
shap_values_hgb = explainer_hgb(X_test_scaled_sample)
shap.summary_plot(shap_values_hgb, X_test.iloc[sample_idx], feature_names=X.columns, plot_type="bar")
plt.title("SHAP Summary - HistGradientBoosting")
plt.tight_layout()
plt.savefig("shap_hgb_summary.png", dpi=150)
plt.close()

explainer_lgb = shap.Explainer(lgb_model, X_train_scaled)
shap_values_lgb = explainer_lgb(X_test_scaled_sample)
shap.summary_plot(shap_values_lgb, X_test.iloc[sample_idx], feature_names=X.columns, plot_type="bar")
plt.title("SHAP Summary - LightGBM")
plt.tight_layout()
plt.savefig("shap_lgb_summary.png", dpi=150)
plt.close()

explainer_xgb = shap.Explainer(xgb_model, X_train_scaled)
shap_values_xgb = explainer_xgb(X_test_scaled_sample)
shap.summary_plot(shap_values_xgb, X_test.iloc[sample_idx], feature_names=X.columns, plot_type="bar")
plt.title("SHAP Summary - XGBoost")
plt.tight_layout()
plt.savefig("shap_xgb_summary.png", dpi=150)
plt.close()

explainer_cat = shap.Explainer(cat_model, X_train_scaled)
shap_values_cat = explainer_cat(X_test_scaled_sample)
shap.summary_plot(shap_values_cat, X_test.iloc[sample_idx], feature_names=X.columns, plot_type="bar")
plt.title("SHAP Summary - CatBoost")
plt.tight_layout()
plt.savefig("shap_cat_summary.png", dpi=150)
plt.close()
print("*S.T.A.R. training and analysis complete. All outputs saved.")
```